



Triggers no MySQL

1. Introdução às Triggers no MySQL

Triggers são procedimentos automáticos executados em resposta a eventos em uma tabela de um banco de dados. Eles podem ser ativados por comandos `INSERT`, `UPDATE` ou `DELETE`, e são úteis para garantir a integridade dos dados, realizar cálculos automáticos ou monitorar modificações no banco.

2. Tipos de Triggers

No MySQL, existem dois tipos principais de triggers:

1. **BEFORE**: Executa antes da operação (inserção, atualização ou exclusão).
2. **AFTER**: Executa após a operação.

Cada trigger é associado a uma tabela específica e a um evento.

3. Sintaxe Básica de Criação de Triggers

A sintaxe para criar uma trigger no MySQL é a seguinte:

```
DELIMITER $$  
CREATE TRIGGER trigger_name  
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}  
ON table_name  
FOR EACH ROW  
BEGIN  
    -- Comandos SQL a serem executados  
END$$  
DELIMITER;
```

- **trigger_name**: Nome da trigger.
- **BEFORE** ou **AFTER**: Define se o trigger executa antes ou depois do evento.
- **INSERT**, **UPDATE** ou **DELETE**: Define qual evento aciona o trigger.
- **table_name**: A tabela associada ao trigger.
- **FOR EACH ROW**: Executa o código para cada linha afetada pela operação.

Remover Trigger

Para **apagar (remover) uma trigger** no MySQL, você pode usar o comando `DROP TRIGGER`. A sintaxe básica é a seguinte:

```
DROP TRIGGER [IF EXISTS] trigger_name;
```

Parâmetros:

- `[IF EXISTS]`: Opcional. Previne o erro caso a trigger não exista.
- `trigger_name`: O nome da trigger que você deseja apagar.

Exemplo:

Se você quiser apagar uma trigger chamada `after_insert_usuarios`, o comando seria:

```
DROP TRIGGER IF EXISTS after_insert_usuarios;
```

Esse comando remove a trigger chamada `after_insert_usuarios` da tabela associada.

Observações:

- Antes de executar o comando, certifique-se de que o nome da trigger está correto, pois ele é específico para a tabela e o tipo de evento associado.
- **As triggers são definidas por tabela**, então, mesmo que existam várias triggers com nomes semelhantes em diferentes tabelas, o MySQL trata cada uma de forma separada.

4. Exemplos Práticos

Exemplo 1: Trigger BEFORE INSERT

Este exemplo cria uma trigger que, antes de inserir uma nova linha, ajusta o valor de um campo `nome` para maiúsculas.

```
CREATE TRIGGER before_insert_uppercase_name
BEFORE INSERT ON usuarios
FOR EACH ROW
BEGIN
    SET NEW.nome = UPPER(NEW.nome);
END;
```

- **Objetivo:** Toda vez que um novo usuário for inserido na tabela `usuarios`, o nome será armazenado em letras maiúsculas.

```
CREATE TABLE usuarios (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL  
);  
  
INSERT INTO usuarios (nome, email)  
VALUES  
('Alice Silva', 'alice.silva@email.com'),  
('Bruno Oliveira', 'bruno.oliveira@email.com'),  
('Carla Pereira', 'carla.pereira@email.com'),  
('Diego Souza', 'diego.souza@email.com'),  
('Fernanda Costa', 'fernanda.costa@email.com');
```

Exemplo 2: Trigger AFTER UPDATE

Este exemplo cria uma trigger que registra todas as alterações feitas na tabela de `produtos` em uma tabela de log.

```
CREATE TRIGGER after_update_produto
AFTER UPDATE ON produtos
FOR EACH ROW
BEGIN
    INSERT INTO log_alteracoes (produto_id, alterado_em, operacao)
    VALUES (NEW.id, NOW(), "Update");
END;
```

- **Objetivo:** Toda vez que um produto for atualizado, o `produto_id` e a data/hora da alteração são registrados em uma tabela de log chamada `log_alteracoes`.

```
CREATE TABLE produtos (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100),  
    categoria VARCHAR(50),  
    preco DECIMAL(10, 2)  
);  
  
CREATE TABLE log_alteracoes (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    produto_id INT,  
    alterado_em DATETIME NOT NULL,  
    operacao VARCHAR(20) NOT NULL,  
    descricao VARCHAR(255),  
    FOREIGN KEY (produto_id) REFERENCES produtos(id)  
);
```

```
INSERT INTO produtos (nome, categoria, preco)
VALUES
('Notebook Dell Inspiron', 'Eletrônicos', 3500.00),
('Smartphone Samsung Galaxy', 'Eletrônicos', 1200.00),
('Fone de Ouvido JBL', 'Acessórios', 250.50),
('Smart TV LG 55"', 'Eletrônicos', 3200.99),
('Cadeira Gamer', 'Móveis', 800.00),
('Mouse Gamer Razer', 'Acessórios', 350.00),
('Teclado Mecânico Corsair', 'Acessórios', 550.75),
('Geladeira Brastemp Frost Free', 'Eletrodomésticos', 2300.00),
('Micro-ondas Philco', 'Eletrodomésticos', 450.00),
('Aspirador de Pó Electrolux', 'Eletrodomésticos', 600.00);

update produtos set preco = 700 where id = 10;
```

5. Trabalhando com os Pseudoregistros OLD e NEW

Dentro de uma trigger, você pode acessar os valores dos campos antigos (antes da operação) e novos (após a operação) usando os pseudoregistros **OLD** e **NEW**:

- **OLD**: Representa o valor da coluna antes da operação.
- **NEW**: Representa o valor da coluna após a operação.

Exemplo 3: Trigger para Monitorar Alterações de Salário

Neste exemplo, queremos manter um histórico das mudanças de salário de funcionários.

```
CREATE TRIGGER after_update_salario
AFTER UPDATE ON funcionarios
FOR EACH ROW
BEGIN
    IF OLD.salario <> NEW.salario THEN
        INSERT INTO historico_salarios (funcionario_id, salario_antigo, salario_novo, alterado_em)
        VALUES (OLD.id, OLD.salario, NEW.salario, NOW());
    END IF;
END;
```

- **Objetivo:** Registrar no histórico apenas quando houver mudanças no salário de um funcionário.

6. Exercícios Práticos

Exercício 1:

Crie uma trigger que, ao deletar um cliente da tabela `clientes`, insira uma linha na tabela `clientes_excluidos` com o `id` e a data da exclusão.

Solução: Esta trigger registra a exclusão de um cliente, salvando o `cliente_id` e o horário da exclusão.

```
CREATE TRIGGER after_delete_cliente
AFTER DELETE ON clientes
FOR EACH ROW
BEGIN
    INSERT INTO clientes_excluidos (cliente_id, excluido_em)
    VALUES (OLD.id, NOW());
END;
```

Exercício 2:

Crie uma trigger que impede a inserção de novos produtos na tabela `produtos` caso o preço seja menor que 10.

Solução: Esta trigger valida se o preço é maior ou igual a 10, gerando um erro caso contrário.

```
CREATE TRIGGER before_insert_produto
BEFORE INSERT ON produtos
FOR EACH ROW
BEGIN
    IF NEW.preco < 10 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'O preço do produto deve ser maior ou igual a 10.';
    END IF;
END;
```

7. Boas Práticas ao Utilizar Triggers

1. **Manter as Triggers Simples:** Evite adicionar lógica complexa. Triggers muito complexos podem afetar o desempenho.
2. **Cuidado com Loops:** Evite triggers que possam desencadear loops infinitos. Exemplo: um trigger `AFTER UPDATE` que faz um novo `UPDATE` na mesma tabela.
3. **Monitorar o Impacto no Desempenho:** Triggers executam para cada linha afetada, então em operações em massa, podem impactar a performance.

8. Conclusão

Triggers são uma ferramenta poderosa no MySQL para automatizar tarefas e garantir a integridade dos dados. Contudo, é importante usá-las com cuidado, garantindo que não causem efeitos indesejados no desempenho do banco de dados.

Exercício 3: Verificação de Idade Mínima para Cadastro

Descrição: Você foi encarregado de garantir que o sistema não permita cadastrar um usuário menor de 18 anos. Crie uma trigger que verifique, antes da inserção, se a data de nascimento do usuário indica que ele tem pelo menos 18 anos.

Requisitos:

- A tabela `usuarios` tem os campos `id`, `nome` e `data_nascimento`.
- A trigger deve impedir a inserção de um usuário menor de 18 anos, lançando um erro.

Tabela exemplo:

```
CREATE TABLE usuarios (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100),  
  data_nascimento DATE  
);
```

Exercício 4: Registro de Alterações de Preço de Produtos

Descrição: Você está desenvolvendo um sistema de controle de estoque e precisa registrar toda alteração de preço dos produtos. Sempre que o preço de um produto for alterado, você deve armazenar o preço antigo, o novo preço, e a data da alteração em uma tabela de histórico.

Requisitos:

- A tabela `produtos` tem os campos `id`, `nome` e `preco`.
- A tabela `historico_precos` tem os campos `produto_id`, `preco_antigo`, `preco_novo`, e `data_alteracao`.
- A trigger deve ser acionada após a atualização de qualquer produto, registrando as alterações de preço.

Tabelas exemplo:

```
CREATE TABLE produtos (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100),  
    preco DECIMAL(10, 2)  
);  
  
CREATE TABLE historico_precos (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    produto_id INT,  
    preco_antigo DECIMAL(10, 2),  
    preco_novo DECIMAL(10, 2),  
    data_alteracao DATETIME,  
    FOREIGN KEY (produto_id) REFERENCES produtos(id)  
);
```

Exercício 5: Registro de Deleção de Funcionários

Descrição: No seu sistema de RH, é necessário manter um registro de todos os funcionários que foram excluídos da tabela `funcionarios`. Esse registro deve conter o `id`, o `nome` do funcionário excluído, e a data da exclusão.

Requisitos:

- A tabela `funcionarios` tem os campos `id`, `nome`, `cargo` e `salario`.
- A tabela `funcionarios_excluidos` deve armazenar o `id`, `nome`, e a data da exclusão do funcionário.
- A trigger deve ser acionada após a exclusão de qualquer funcionário da tabela `funcionarios`, armazenando essas informações na tabela de `funcionarios_excluidos`.

Tabelas exemplo:

```
CREATE TABLE funcionarios (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100),  
    cargo VARCHAR(50),  
    salario DECIMAL(10, 2)  
);
```

```
CREATE TABLE funcionarios_excluidos (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    funcionario_id INT,  
    nome VARCHAR(100),  
    data_exclusao DATETIME  
);
```